Mongoose Advanced Notes (Production Level)

এই ডকুমেন্টটা **শুধু Mongoose** নিয়ে — MongoDB shell না, pure **Node.js + Mongoose (ODM)**। লক্ষ্য: **Real project, SaaS, Interview, Production** সব জায়গায় কাজে লাগবে।

1️⃣ Mongoose আবার একটু Clear করি

Mongoose হলো **ODM (Object Data Modeling)** লাইব্রেরি।

👉 MongoDB raw database 👉 Mongoose = structure + rule + power

কেন Mongoose লাগে? - Schema enforce করতে - Validation দিতে - Relation handle করতে - Middleware / hook চালাতে - Clean architecture বানাতে

2️⃣ Advanced Schema Design

Full-feature Schema

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  role: { type: String, enum: ['admin', 'user'], default: 'user' },
  isActive: { type: Boolean, default: true }
}, { timestamps: true });
```

👉 Production-ready schema

3️⃣ Indexing (Very Important)

```
userSchema.index({ email: 1 });
userSchema.index({ role: 1, createdAt: -1 });
```

👉 Large data তে performance বাঁচায়

4️⃣ Virtual Fields

```
userSchema.virtual('info').get(function () {
  return `${this.name} - ${this.email}`;
});
```

👉 DB তে save হয় না, response enrich করে

5️⃣ Instance vs Static Method

Instance Method

```
userSchema.methods.isAdmin = function () {
  return this.role === 'admin';
};
```

Static Method

```
userSchema.statics.findByEmail = function (email) {
  return this.findOne({ email });
};
```

👉 Business logic clean থাকে

6️⃣ Middleware (Hooks) – Power Zone ☑️

Pre Save (Password hash type case)

```
userSchema.pre('save', function (next) {
  console.log('Before save');
  next();
});
```

Pre Find (Global filter)

```
userSchema.pre(/^find/, function (next) {
  this.where({ isActive: true });
```

```
  next();
});
```

👉 Soft delete, auto-filter system

7️⃣ Relationship & Populate

Reference

```
user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
```

Populate

```
Post.find().populate('user');
```

Controlled Populate

```
Post.find().populate({
  path: 'user',
  select: 'name email'
});
```

👉 Over-fetching avoid

8️⃣ Virtual Populate (Advanced)

```
userSchema.virtual('posts', {
  ref: 'Post',
  localField: '_id',
  foreignField: 'user'
});
```

```
User.find().populate('posts');
```

👉 No duplicate data, clean relation

9️⃣ Lean Query (Performance)

```
User.find().lean();
```

```
User.find().lean({ virtuals: true });
```

👉 Fast response, low memory

🔟 Transactions (Money-safe)

```
const session = await mongoose.startSession();
await session.startTransaction();

try {
  await Order.create([order], { session });
  await Payment.create([payment], { session });
  await session.commitTransaction();
} catch (err) {
  await session.abortTransaction();
}
```

👉 Order + Payment atomic

1️⃣1️⃣ Plugin Architecture (SaaS Style)

```
function softDelete(schema) {
  schema.add({ deleted: { type: Boolean, default: false } });
}

userSchema.plugin(softDelete);
```

👉 Reusable logic

1️⃣2️⃣ Discriminator (Inheritance)

```
const baseSchema = new Schema({ name: String }, { discriminatorKey: 'type' });
const User = mongoose.model('User', baseSchema);
```

```
const Admin = User.discriminator('Admin', new Schema({ level: Number }));
```

👉 Multiple user type

1️⃣3️⃣ Pagination Pattern

```
schema.statics.paginate = async function (filter, page = 1, limit = 10) {
  const skip = (page - 1) * limit;
  const data = await this.find(filter).skip(skip).limit(limit);
  const total = await this.countDocuments(filter);
  return { data, total, page, limit };
};
```

1️⃣4️⃣ Error Handling (Production)

```
schema.post('save', function (error, doc, next) {
  if (error.code === 11000) {
    next(new Error('Duplicate key error'));
  } else {
    next(error);
  }
});
```

1️⃣5️⃣ Clean Folder Structure

```
models/
 ├── user.model.js
 ├── post.model.js
 ├── plugins/
 └── index.js
```

🔝 Final Outcome

এই Advanced Mongoose শেষ করলে তুমি: - Production-grade backend লিখতে পারবে - SaaS & large project handle করতে পারবে - MongoDB + Mongoose relation deep বুঝবে - Interview confidently দিতে পারবে

👉 Next Step (তুমি যেটা চাও): - Mongoose + Express Full Project Structure - Auth System (JWT + RBAC) - Advanced Aggregation with Mongoose